HDOC-33

Aim:

To write a menu-driven C program using separate functions and variable-length 2D arrays (VLAs) to:

1. Verify the associative property of matrix addition:
   (A + B) + C = A + (B + C)
2. Verify the associative property of matrix multiplication:
   (A × B) × C = A × (B × C)
3. Exit the program

Task 1: Algorithm:

1. Start the program.
2. Display the menu:
   - Matrix Addition Associativity
   - Matrix Multiplication Associativity
   - Exit
3. Read the user's choice.
4. **For addition:**
   - Input m and n.
   - Declare matrices A, B, and C of size m × n.
   - Input all three matrices.
   - Calculate A + B.
   - Calculate (A + B) + C.
   - Calculate B + C.
   - Calculate A + (B + C).

- Display both final matrices.
- Compare them.
- If equal, associative property is verified.

5. **For multiplication:**
    - Input m, n, p, and q.
    - Declare:
        - A as m × n
        - B as n × p
        - C as p × q
    - Calculate A × B, which gives an m × p matrix.
    - Calculate (A × B) × C, which gives an m × q matrix.
    - Calculate B × C, which gives an n × q matrix.
    - Calculate A × (B × C), which gives an m × q matrix.
    - Display both final matrices.
    - Compare them.
    - If equal, associative property is verified.
6. If choice is 3, exit.
7. Otherwise, display "Invalid choice".
8. Stop.

Task 2: Test case table:

Test Case	Choice	Dimensions	Matrix A	Matrix B	Matrix C	Expected Result	Status
1	1	**m=2, n=2**	1 2; 3 4	5 6; 7 8	1 1; 1 1	(A+B)+C = A+(B+C) = [7 9; 11 13] → **Associative property verified**	Pass
2	1	**m=2, n=3**	1 2 3; 4 5 6	2 3 4; 5 6 7	3 2 1; 1 2 3	Both sides produce the same **2×3 matrix** → **Associative property verified**	Pass
3	2	**m=2, n=2, p=2, q=2**	1 2; 3 4	5 6; 7 8	1 0; 0 1	(A×B)×C = A×(B×C) = [19 22; 43 50] → **Associative property verified**	Pass
4	2	**m=2, n=3, p=2, q=2**	1 2 3; 4 5 6	1 2; 3 4; 5 6	1 2; 3 4	Both (A×B)×C and A×(B×C) give **2×2** matrix [50 72; 113 162] → **Associative property verified**	Pass
5	3	—	—	—	—	Displays **"Exiting program..."** and terminates the program	Pass

Task 3: C program:

```
#include <stdio.h>
/* Function to input a matrix */
void inputMatrix(int rows, int cols, int M[rows][cols])
{
   int i, j;
   for (i = 0; i < rows; i++)
   {
      for (j = 0; j < cols; j++)
      {
         scanf("%d", &M[i][j]);
      }
   }
}
/* Function to display a matrix */
void displayMatrix(int rows, int cols, int M[rows][cols])
{
   int i, j;
   for (i = 0; i < rows; i++)
   {
      for (j = 0; j < cols; j++)
      {
         printf("%d\t", M[i][j]);
      }
      printf("\n");
   }
}
/* Function to add two matrices */
void addMatrix(int rows, int cols,
            int A[rows][cols],
            int B[rows][cols],
            int result[rows][cols])
{
   int i, j;
   for (i = 0; i < rows; i++)
   {
      for (j = 0; j < cols; j++)
      {
         result[i][j] = A[i][j] + B[i][j];
      }
   }
}
/* Function to multiply two matrices */
void multiplyMatrix(int r1, int c1, int c2,
                int A[r1][c1],
                int B[c1][c2],
```

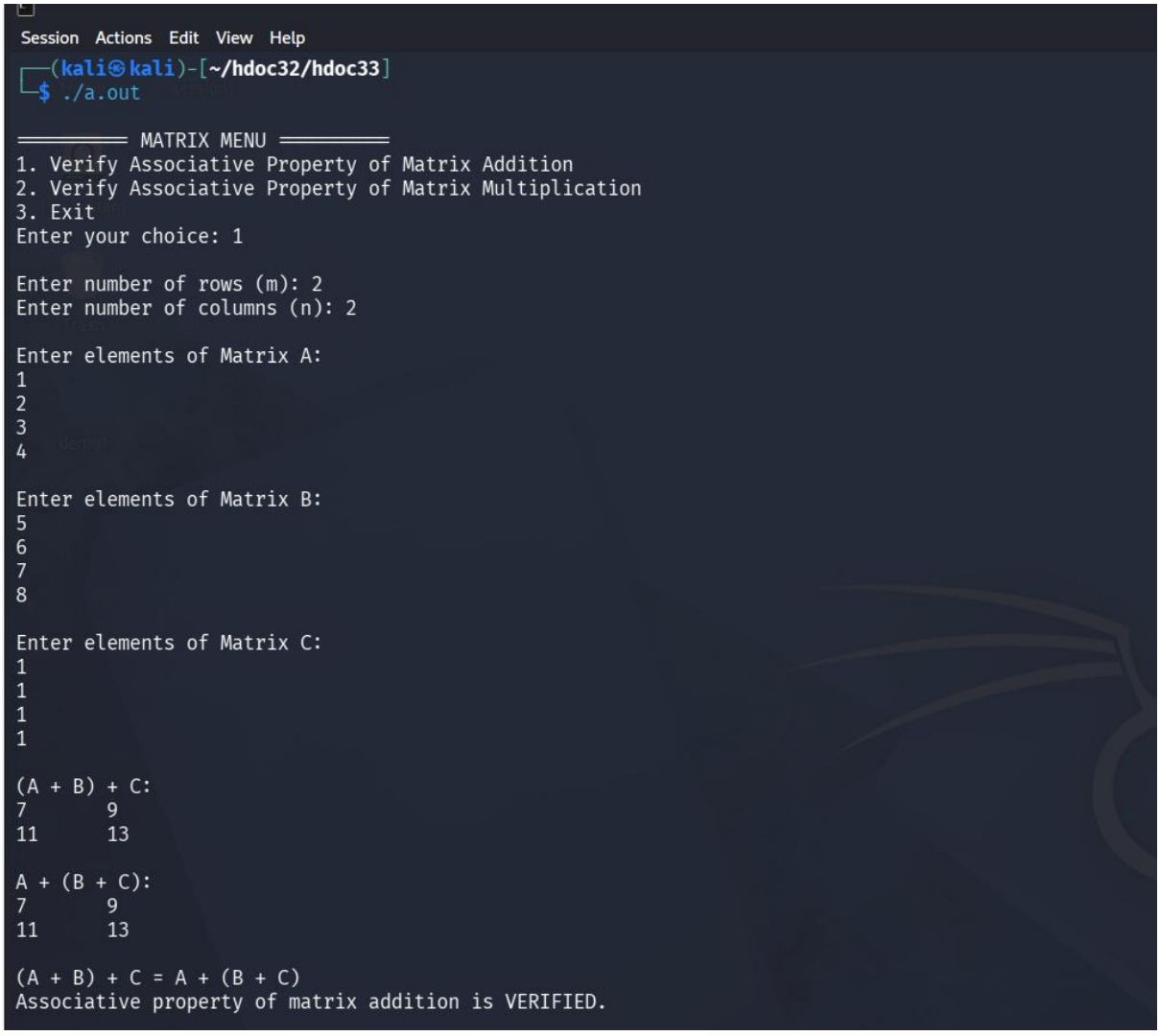
```
            int result[r1][c2])
{
  int i, j, k;
  for (i = 0; i < r1; i++)
  {
    for (j = 0; j < c2; j++)
    {
      result[i][j] = 0;
      for (k = 0; k < c1; k++)
      {
        result[i][j] += A[i][k] * B[k][j];
      }
    }
  }
}
/* Function to compare two matrices */
int compareMatrix(int rows, int cols,
            int A[rows][cols],
            int B[rows][cols])
{
  int i, j;
  for (i = 0; i < rows; i++)
  {
    for (j = 0; j < cols; j++)
    {
      if (A[i][j] != B[i][j])
      {
        return 0;
      }
    }
  }
  return 1;
}
int main()
{
  int choice;
  do
  {
    printf("\n========== MATRIX MENU ==========\n");
    printf("1. Verify Associative Property of Matrix Addition\n");
    printf("2. Verify Associative Property of Matrix Multiplication\n");
    printf("3. Exit\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);
    switch (choice)
    {
      /* ASSOCIATIVE PROPERTY OF ADDITION */
```

```
case 1:
{
   int m, n;
   printf("\nEnter number of rows (m): ");
   scanf("%d", &m);
   printf("Enter number of columns (n): ");
   scanf("%d", &n);
   int A[m][n];
   int B[m][n];
   int C[m][n];
   int AB[m][n];
   int BC[m][n];
   int left[m][n];
   int right[m][n];
   printf("\nEnter elements of Matrix A:\n");
   inputMatrix(m, n, A);
   printf("\nEnter elements of Matrix B:\n");
   inputMatrix(m, n, B);
   printf("\nEnter elements of Matrix C:\n");
   inputMatrix(m, n, C);
   /* Left side: (A + B) + C */
   addMatrix(m, n, A, B, AB);
   addMatrix(m, n, AB, C, left);
   /* Right side: A + (B + C) */
   addMatrix(m, n, B, C, BC);
   addMatrix(m, n, A, BC, right);
   printf("\n(A + B) + C:\n");
   displayMatrix(m, n, left);
   printf("\nA + (B + C):\n");
   displayMatrix(m, n, right);
   if (compareMatrix(m, n, left, right))
   {
      printf("\n(A + B) + C = A + (B + C)\n");
      printf("Associative property of matrix addition "
           "is VERIFIED.\n");
   }
   else
   {
      printf("\nAssociative property of matrix addition "
           "is NOT verified.\n");
   }
   break;
}
/* ASSOCIATIVE PROPERTY OF MULTIPLICATION */
case 2:
{
   int m, n, p, q;
```

```
    printf("\nEnter m: ");
    scanf("%d", &m);
    printf("Enter n: ");
    scanf("%d", &n);
    printf("Enter p: ");
    scanf("%d", &p);
    printf("Enter q: ");
    scanf("%d", &q);
    int A[m][n];
    int B[n][p];
    int C[p][q];
    int AB[m][p];
    int BC[n][q];
    int left[m][q];
    int right[m][q];
    printf("\nEnter elements of Matrix A (%d x %d):\n",
        m, n);
    inputMatrix(m, n, A);
    printf("\nEnter elements of Matrix B (%d x %d):\n",
        n, p);
    inputMatrix(n, p, B);
    printf("\nEnter elements of Matrix C (%d x %d):\n",
        p, q);
    inputMatrix(p, q, C);
    /* Left side: (A x B) x C */
    multiplyMatrix(m, n, p, A, B, AB);
    multiplyMatrix(m, p, q, AB, C, left);
    /* Right side: A x (B x C) */
    multiplyMatrix(n, p, q, B, C, BC);
    multiplyMatrix(m, n, q, A, BC, right);
    printf("\n(A x B) x C:\n");
    displayMatrix(m, q, left);
    printf("\nA x (B x C):\n");
    displayMatrix(m, q, right);
    if (compareMatrix(m, q, left, right))
    {
      printf("\n(A x B) x C = A x (B x C)\n");
      printf("Associative property of matrix multiplication "
          "is VERIFIED.\n");
    }
    else
    {
      printf("\nAssociative property of matrix multiplication "
          "is NOT verified.\n");
    }
    break;
}
```

```
        case 3:
        {
            printf("\nExiting program...\n");
            break;
        }
        default:
        {
            printf("\nInvalid choice! Please enter 1, 2 or 3.\n");
        }
    }
  } while (choice != 3);
  return 0;
}
```

Task 4: Output:



```
========== MATRIX MENU ==========
1. Verify Associative Property of Matrix Addition
2. Verify Associative Property of Matrix Multiplication
3. Exit
Enter your choice: 1

Enter number of rows (m): 2
Enter number of columns (n): 3

Enter elements of Matrix A:
1
2
3
4
5
6

Enter elements of Matrix B:
2
3
4
5
6
7

Enter elements of Matrix C:
3
2
1
1
2
3

(A + B) + C:
6       7       8
10      13      16

A + (B + C):
6       7       8
10      13      16

(A + B) + C = A + (B + C)
Associative property of matrix addition is VERIFIED.
```

```
═══════ MATRIX MENU ═══════
1. Verify Associative Property of Matrix Addition
2. Verify Associative Property of Matrix Multiplication
3. Exit
Enter your choice: 2

Enter m: 2
Enter n: 2
Enter p: 2
Enter q: 2

Enter elements of Matrix A (2 x 2):
1
2
3
4

Enter elements of Matrix B (2 x 2):
5
6
7
8

Enter elements of Matrix C (2 x 2):
1
0
0
1

(A x B) x C:
19      22
43      50

A x (B x C):
19      22
43      50

(A x B) x C = A x (B x C)
Associative property of matrix multiplication is VERIFIED.
```

```
========== MATRIX MENU ==========
1. Verify Associative Property of Matrix Addition
2. Verify Associative Property of Matrix Multiplication
3. Exit
Enter your choice: 2

Enter m: 2
Enter n: 3
Enter p: 2
Enter q: 2

Enter elements of Matrix A (2 x 3):
1
2
3
4
5
6

Enter elements of Matrix B (3 x 2):
1
2
3
4
5
6

Enter elements of Matrix C (2 x 2):
1
2
3
4

(A x B) x C:
106     156
241     354

A x (B x C):
106     156
241     354

(A x B) x C = A x (B x C)
Associative property of matrix multiplication is VERIFIED.

========== MATRIX MENU ==========
1. Verify Associative Property of Matrix Addition
2. Verify Associative Property of Matrix Multiplication
3. Exit
Enter your choice: 3

Exiting program...
```